

SQL: Data Manipulation

Lecture 05

Borough of Manhattan Community College

Introduction to SQL

- ❑ Over the last few years, Structured Query Language (SQL) has become the standard relational database language.
- ❑ More than one hundred DBMSs now support SQL, running on various hardware platforms from PCs to mainframes.
- ❑ As a language, the ISO SQL standard has two major components:
 - a Data Definition Language (DDL) for defining the database structure and controlling access to the data;
 - a Data Manipulation Language (DML) for retrieving and updating data.

Objectives of Database Language

- ❑ Ideally, a database language should allow a user to:
 - create the database and relation structures;
 - perform basic data management tasks, such as the insertion, modification, and deletion of data from the relations;
 - perform both simple and complex queries.
- ❑ A database language must perform these tasks with minimal user effort, and its command structure and syntax must be relatively easy to learn.
- ❑ Finally, the language must be portable: we can use the same command structure and syntax when we move from one DBMS to another.
- ❑ SQL is intended to satisfy these requirements.

Writing SQL Commands

□ SQL is a relatively easy language to learn:

- The command structure consists of standard English words such as CREATE TABLE, INSERT, SELECT.
- It is a nonprocedural language; you specify ***what*** information you require, rather than ***how*** to get it. In other words, SQL does not require you to specify the access methods to the data.
- Like most modern languages, SQL is essentially free-format, which means that parts of statements do not have to be typed at particular locations on the screen.

Writing SQL Commands

- ❑ An SQL statement consists of **reserved words** and **user-defined words**:
 - Reserved words are a fixed part of the SQL language and have a fixed meaning.
 - User-defined words are made up by the user and represent the names of various database objects such as tables, columns, views, indexes, and so on.
- ❑ Most components of an SQL statement are **case-insensitive**, which means that letters can be typed in either upper or lowercase.
- ❑ The one important exception is that literal character data must be typed exactly as it appears in the database. For example, if we store a person's surname as **'SMITH'** and then search for it using the string **'Smith'** the row will not be found.

Writing SQL Commands

- ❑ Throughout this and the next chapter, we use the following extended form of the Backus Naur Form (BNF) notation to define SQL statements:
 - uppercase letters are used to represent reserved words and must be spelled exactly as shown;
 - lowercase letters are used to represent user-defined words;
 - a vertical bar (|) indicates a **choice** among alternatives; for example, a | b | c;
 - curly braces indicate a **required element**; for example, {a};
 - square brackets indicate an **optional element**; for example, [a];
 - an ellipsis (. . .) is used to indicate **optional repetition** of an item zero or more times.

- ❑ For example: {a|b} (, c . . .) means:
either **a** or **b** followed by zero or more repetitions of **c** separated by commas.

Data Manipulation

- ❑ This section looks at the following SQL DML statements:
 - SELECT – to query data in the database
 - INSERT – to insert data into a table
 - UPDATE – to update data in a table
 - DELETE – to delete data from a table

- ❑ **Literals** are constants that are used in SQL statements. There are different forms of literals for every data type supported by SQL.

- ❑ For simplicity, we can distinguish between literals that are enclosed in single quotes and those that are not as follows:
 - All nonnumeric data values must be enclosed in single quotes;
 - All numeric data values must not be enclosed in single quotes.

Simple Queries

- ❑ The purpose of the SELECT statement is to retrieve and display data from one or more database tables.
- ❑ SELECT is the most frequently used SQL command and has the following general form:

```
SELECT      [DISTINCT | ALL] { * | columnExpression [ AS newName] [, . . .] }  
FROM       TableName [alias] [, . . .]  
[WHERE      condition]  
[GROUP BY  columnList] [HAVING condition]  
[ORDER BY  columnList]
```

- *columnExpression* represents a column name or an expression,
- *TableName* is the name of an existing database table or view,
- *alias* is an optional abbreviation for *TableName*.

Simple Queries

- ❑ The sequence of processing in a SELECT statement is:
 - SELECT specifies which columns are to appear in the output
 - FROM specifies the table or tables to be used
 - WHERE filters the rows subject to some condition
 - GROUP BY forms groups of rows with the same column value
 - HAVING filters the groups subject to some condition
 - ORDER BY specifies the order of the output
- ❑ The order of the clauses in the SELECT statement cannot be changed.
- ❑ The only two mandatory clauses are SELECT and FROM.

Simple Queries

EXAMPLE Retrieve all columns, all rows

List full details of all staff.

```
SELECT staffNo, fName, lName, position, sex, DOB, salary, branchNo  
FROM Staff;
```

The following statement is an equivalent and shorter way of expressing this query:

```
SELECT *  
FROM Staff;
```

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

Simple Queries

EXAMPLE **Retrieve specific columns, all rows**

Produce a list of salaries for all staff, showing only the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary  
FROM Staff;
```

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG37	Ann	Beech	12000.00
SG14	David	Ford	18000.00
SA9	Mary	Howe	9000.00
SG5	Susan	Brand	24000.00
SL41	Julie	Lee	9000.00

Simple Queries

EXAMPLE Use of DISTINCT

List the property numbers of all properties that have been viewed.

```
SELECT propertyNo  
FROM Viewing;
```

propertyNo

PA14

PG4

PG4

PA14

PG36

To eliminate the duplicates, we use the DISTINCT keyword.

```
SELECT DISTINCT propertyNo  
FROM Viewing;
```

propertyNo

PA14

PG4

PG36

Simple Queries

EXAMPLE Calculated fields

Produce a list of monthly salaries for all staff, showing the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary/12  
FROM Staff;
```

staffNo	fName	lName	col4
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

The ISO standard allows the column to be named using an AS clause.

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary  
FROM Staff;
```

Simple Queries

Row selection (**WHERE** clause)

The previous examples show the use of the SELECT statement to retrieve all rows from a table. However, we often need to restrict the rows that are retrieved. This can be achieved with the WHERE clause, which consists of the keyword WHERE followed by a search condition that specifies the rows to be retrieved. The five basic search conditions are as follows:

- *Comparison* Compare the value of one expression to the value of another expression.
- *Range* Test whether the value of an expression falls within a specified range of values.
- *Set membership* Test whether the value of an expression equals one of a set of values.
- *Pattern match* Test whether a string matches a specified pattern.
- *Null* Test whether a column has a null (unknown) value.

The WHERE clause is equivalent to the relational algebra Selection operation.

Simple Queries

EXAMPLE **Comparison search condition**

List all staff with a salary greater than £10,000.

```
SELECT staffNo, fName, lName, position, salary  
FROM Staff  
WHERE salary > 10000;
```

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG37	Ann	Beech	Assistant	12000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

Simple Queries

EXAMPLE **Compound comparison search condition**

List the addresses of all branch offices in London or Glasgow.

```
SELECT *  
FROM Branch  
WHERE city = 'London' OR city = 'Glasgow';
```

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

Simple Queries

EXAMPLE Range search condition (**BETWEEN/NOT BETWEEN**)

List all staff with a salary between £20,000 and £30,000.

```
SELECT staffNo, fName, lName, position, salary  
FROM Staff  
WHERE salary BETWEEN 20000 AND 30000;
```

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG5	Susan	Brand	Manager	24000.00

There is also a negated version of the range test (**NOT BETWEEN**) that checks for values outside the range.

We could have expressed the previous query as:

```
SELECT staffNo, fName, lName, position, salary  
FROM Staff  
WHERE salary >= 20000 AND salary <= 30000;
```

Simple Queries

EXAMPLE **Set membership search condition (IN/NOT IN)**

List all managers and supervisors.

```
SELECT staffNo, fName, lName, position  
FROM Staff  
WHERE position IN ('Manager', 'Supervisor');
```

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

There is a negated version (NOT IN) that can be used to check for data values that do not lie in a specific list of values.

We could have expressed the previous query as:

```
SELECT staffNo, fName, lName, position  
FROM Staff  
WHERE position = 'Manager' OR position = 'Supervisor';
```

Simple Queries

EXAMPLE Pattern match search condition (LIKE/NOT LIKE)

Find all owners with the string 'Glasgow' in their address.

For this query, we must search for the string 'Glasgow' appearing somewhere within the address column of the PrivateOwner table.

SQL has two special pattern-matching symbols:

- The % percent character represents any sequence of zero or more characters (*wildcard*).
- The _ underscore character represents any single character.

For example:

- address LIKE 'H%' means the first character must be *H*, but the rest of the string can be anything.
- address LIKE 'H_ _ _' means that there must be exactly four characters in the string, the first of which must be an *H*.
- address LIKE '%e' means any sequence of characters, of length at least 1, with the last character an *e*.
- address LIKE '%Glasgow%' means a sequence of characters of any length containing *Glasgow*.
- address NOT LIKE 'H%' means the first character cannot be an *H*.

Simple Queries

EXAMPLE Pattern match search condition (**LIKE**/**NOT LIKE**)

Find all owners with the string 'Glasgow' in their address.

```
SELECT ownerNo, fName, lName, address, telNo  
FROM PrivateOwner  
WHERE address LIKE '%Glasgow%';
```

ownerNo	fName	lName	address	telNo
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

Simple Queries

EXAMPLE **NULL search condition (IS NULL/IS NOT NULL)**

List the details of all viewings on property PG4 where a comment has not been supplied.

Viewing

clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-13	too small
CR76	PG4	20-Apr-13	too remote
CR56	PG4	26-May-13	
CR62	PA14	14-May-13	no dining room
CR56	PG36	28-Apr-13	

```
SELECT clientNo, viewDate
FROM Viewing
WHERE propertyNo = 'PG4' AND comment IS NULL;
```

clientNo	viewDate
CR56	26-May-13

The negated version (IS NOT NULL) can be used to test for values that are not null.

Sorting Results (ORDER BY Clause)

- ❑ In general, the rows of an SQL query result table are not arranged.
- ❑ However, we can ensure the results of a query are sorted using the ORDER BY clause in the SELECT statement:
 - The ORDER BY clause consists of a list of **column identifiers** that the result is to be sorted on, separated by commas.
 - A column identifier may be either a column name or a column number that identifies an element of the SELECT list by its position within the list.
 - The ORDER BY clause allows the retrieved rows to be ordered in ascending (ASC) or descending (DESC) order on any column or combination of columns.
 - The ORDER BY clause must be the last clause of the SELECT statement.

Sorting Results (ORDER BY Clause)

EXAMPLE **Single-column ordering**

Produce a list of salaries for all staff, arranged in descending order of salary.

```
SELECT staffNo, fName, lName, salary  
FROM Staff  
ORDER BY salary DESC;
```

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG5	Susan	Brand	24000.00
SG14	David	Ford	18000.00
SG37	Ann	Beech	12000.00
SA9	Mary	Howe	9000.00
SL41	Julie	Lee	9000.00

Note that we could have expressed the ORDER BY clause as: ORDER BY 4 DESC, with the 4 relating to the fourth column name in the SELECT list, namely salary.

Sorting Results (ORDER BY Clause)

EXAMPLE **Multiple column ordering**

Produce an abbreviated list of properties arranged in order of property type.

```
SELECT propertyNo, type, rooms, rent  
FROM PropertyForRent  
ORDER BY type;
```

propertyNo	type	rooms	rent
PL94	Flat	4	400
PG4	Flat	3	350
PG36	Flat	3	375
PG16	Flat	4	450
PA14	House	6	650
PG21	House	5	600

There are four flats in this list. As we did not specify any minor sort key, the system arranges these rows in any order it chooses.

Sorting Results (ORDER BY Clause)

To arrange the properties in order of rent, we specify a minor order, as follows:

```
SELECT propertyNo, type, rooms, rent  
FROM PropertyForRent  
ORDER BY type, rent DESC;
```

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

The ISO standard specifies that nulls in a column or expression sorted with ORDER BY should be treated as either less than all nonnull values or greater than all nonnull values. The choice is left to the DBMS implementor.

Using the SQL Aggregate Functions

- ❑ As well as retrieving rows and columns from the database, we often want to perform some form of summation or aggregation of data, similar to the totals at the bottom of a report.
- ❑ The ISO standard defines five aggregate functions:
 - COUNT – returns the number of values in a specified column
 - SUM – returns the sum of the values in a specified column
 - AVG – returns the average of the values in a specified column
 - MIN – returns the smallest value in a specified column
 - MAX – returns the largest value in a specified column
- ❑ COUNT, MIN, and MAX apply to both numeric and nonnumeric fields, but SUM and AVG must be used on numeric fields only.

Using the SQL Aggregate Functions

- ❑ Each aggregate function eliminates nulls first and operates only on the remaining non-null values.
- ❑ COUNT(*) is a special use of COUNT that counts all the rows of a table, regardless of whether nulls or duplicate values occur.
- ❑ If we want to eliminate duplicates before the function is applied, we use the keyword DISTINCT before the column name in the function.
- ❑ DISTINCT has no effect with the MIN and MAX functions. However, it may have an effect on the result of SUM or AVG.

Using the SQL Aggregate Functions

EXAMPLE Use of **COUNT(*)**

How many properties cost more than £350 per month to rent?

```
SELECT COUNT(*) AS myCount  
FROM PropertyForRent  
WHERE rent > 350;
```

myCount

5

EXAMPLE Use of **COUNT(DISTINCT)**

How many different properties were viewed in May 2013?

```
SELECT COUNT(DISTINCT propertyNo) AS myCount  
FROM Viewing  
WHERE viewDate BETWEEN '1-May-13' AND '31-May-13';
```

myCount

2

As the same property may be viewed many times, we have to use the **DISTINCT** keyword to eliminate duplicate properties.

Using the SQL Aggregate Functions

EXAMPLE Use of COUNT and SUM

Find the total number of Managers and the sum of their salaries.

```
SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
WHERE position = 'Manager';
```

myCount	mySum
2	54000.00

EXAMPLE Use of MIN, MAX, AVG

Find the minimum, maximum, and average staff salary.

```
SELECT MIN(salary) AS myMin, MAX(salary) AS myMax, AVG(salary) AS myAvg
FROM Staff;
```

myMin	myMax	myAvg
9000.00	30000.00	17000.00

Grouping Results (GROUP BY Clause)

- ❑ Grouping of data is similar to subtotals in a report. We can use the GROUP BY clause of the SELECT statement to do this.
- ❑ A query that includes the GROUP BY clause groups the data from the SELECT table(s) and produces a single summary row for each group.
- ❑ When GROUP BY is used, each item in the SELECT list must be single-valued per group.
- ❑ In addition, the SELECT clause may contain only:
 - column names;
 - aggregate functions;
 - constants;
 - expressions involving combinations of these elements.

Grouping Results (GROUP BY Clause)

- ❑ All column names in the SELECT list must appear in the GROUP BY clause unless the name is used only in an aggregate function.

EXAMPLE Use of GROUP BY

Find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
ORDER BY branchNo;
```

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

branchNo is not associated with an aggregate function and so must appear in the GROUP BY list.

Grouping Results (GROUP BY Clause)

Conceptually, SQL performs the query as follows:

- (1) SQL divides the staff into groups according to their respective branch numbers. Within each group, all staff have the same branch number. In this example, we get three groups:

branchNo	staffNo	salary		COUNT(staffNo)	SUM(salary)
B003	SG37	12000.00	}	3	54000.00
B003	SG14	18000.00			
B003	SG5	24000.00			
B005	SL21	30000.00	}	2	39000.00
B005	SL41	9000.00			
B007	SA9	9000.00	}	1	9000.00

- (2) For each group, SQL computes the number of staff members and calculates the sum of the values in the **salary** column to get the total of their salaries. SQL generates a single summary row in the query result for each group.
- (3) Finally, the result is sorted in ascending order of branch number, **branchNo**.

Grouping Results (GROUP BY Clause)

- ❑ When the WHERE clause is used with GROUP BY, the WHERE clause is applied first, then groups are formed from the remaining rows that satisfy the search condition.
- ❑ The ISO standard considers two nulls to be equal for purposes of the GROUP BY clause.
- ❑ The HAVING clause is designed for use with the GROUP BY clause to restrict the groups that appear in the final result table.

Grouping Results (GROUP BY Clause)

EXAMPLE Use of HAVING

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum  
FROM Staff  
GROUP BY branchNo  
HAVING COUNT(staffNo) > 1  
ORDER BY branchNo;
```

This is similar to the previous example, with the additional restriction that we want to consider only those groups (that is, branches) with more than one member of staff. This restriction applies to the groups, so the HAVING clause is used.

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00

Grouping Results (GROUP BY Clause)

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
WHERE salary > 10000
GROUP BY branchNo
HAVING COUNT(staffNo) > 1;
```

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

branchNo	myCount	mySum
B003	3	54000.00

Subqueries

- ❑ A **subquery** is the use of a complete SELECT statement embedded within another SELECT statement.
- ❑ A subquery is also called a **nested query**.
- ❑ The results of the **inner** SELECT statement (or **subselect**) are used in the **outer** statement to help determine the contents of the final result.
- ❑ A subselect can be used in the SELECT, the WHERE, and the HAVING clauses of an outer SELECT statement.
- ❑ Subselects may also appear in INSERT, UPDATE, and DELETE statements.

EXAMPLE **Using a subquery with equality**

List the staff who work in the branch at '163 Main St'.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = (SELECT branchNo
                   FROM Branch
                   WHERE street = '163 Main St');
```

The inner **SELECT** statement (**SELECT** branchNo **FROM** Branch . . .) finds the branch number that corresponds to the branch with street name '163 Main St'.

The outer **SELECT** becomes:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = 'B003';
```

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

EXAMPLE **Using a subquery with an aggregate function**

List all staff whose salary is greater than the average salary.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

First, note that we cannot write 'WHERE salary > AVG(salary)', because aggregate functions cannot be used in the WHERE clause. Instead, we use a subquery to find the average salary, and then use the outer SELECT statement to find those staff with a salary greater than this average.

The outer query is reduced then to:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE salary > 17000;
```

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

EXAMPLE Using a subquery with an aggregate function

List all staff whose salary is greater than the average salary, and show by how much their salary is greater than the average.

```
SELECT staffNo, fName, lName, position,  
       salary – (SELECT AVG(salary) FROM Staff) AS salDiff  
FROM Staff  
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

The outer query is reduced then to:

```
SELECT staffNo, fName, lName, position, salary – 17000 AS salDiff  
FROM Staff  
WHERE salary > 17000;
```

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

EXAMPLE Nested subqueries: use of IN

List the properties that are handled by staff who work in the branch at '163 Main St'.

```
SELECT propertyNo, street, city, postcode, type, rooms, rent
FROM PropertyForRent
WHERE staffNo IN (SELECT staffNo
                  FROM Staff
                  WHERE branchNo = (SELECT branchNo
                                    FROM Branch
                                    WHERE street = '163 Main St'));
```

Working from the innermost query outwards, the first query selects the number of the branch at '163 Main St'. The second query then selects those staff who work at this branch number. In this case, there may be more than one such row found, and so we cannot use the equality condition (=) in the outermost query. Instead, we use the IN keyword. The outermost query then retrieves the details of the properties that are managed by each member of staff identified in the middle query.

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

- List the carrierID and the distance of all flights landing at NY state airports.

```
SELECT carrierID, distance FROM Flight
WHERE destination IN (SELECT airportID FROM Airport
                      WHERE state = 'NY');
```

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

- List the names of carriers that have flights landing at France airports.

```
SELECT name FROM Carrier
```

```
WHERE carrierID IN (SELECT carrierID FROM Flight
```

```
WHERE destination IN (SELECT airportID FROM Airport
```

```
WHERE country = 'FRA'));
```

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

- List the carriers that do not leave from JFK.

```
SELECT * FROM Carrier
```

```
WHERE carrierID NOT IN (SELECT carrierID FROM Flight
```

```
WHERE airportID = 'JFK');
```

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

- List the flight(s) with the smallest distance.

```
SELECT * FROM Flight
```

```
WHERE distance = (SELECT MIN(distance) FROM Flight);
```

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

- List the flight(s) with the second smallest distance.

```
SELECT * FROM Flight
```

```
WHERE distance = (SELECT MIN(distance) FROM Flight
```

```
WHERE distance > (SELECT MIN(distance) FROM Flight));
```

Multi-Table Queries

- ❑ To combine columns from several tables into a result table, we need to use a join operation.
- ❑ The SQL join operation combines information from two tables by forming pairs of related rows from the two tables.
- ❑ To perform a join, we simply include more than one table name in the FROM clause, using comma as a separator, and typically including a WHERE clause to specify the join columns.
- ❑ It is also possible to use an alias for a table named in the FROM clause. An alias can be used to qualify a column name whenever there is ambiguity regarding the source of the column name.

Multi-Table Queries

EXAMPLE Simple join

List the names of all clients who have viewed a property, along with any comments supplied.

Client

clientNo	fName	lName	telNo	prefType	maxRent	eMail
CR76	John	Kay	0207-774-5632	Flat	425	john.kay@gmail.com
CR56	Aline	Stewart	0141-848-1825	Flat	350	astewart@hotmail.com
CR74	Mike	Ritchie	01475-392178	House	750	mritchie01@yahoo.co.uk
CR62	Mary	Tregear	01224-196720	Flat	600	maryt@hotmail.co.uk

Viewing

clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-13	too small
CR76	PG4	20-Apr-13	too remote
CR56	PG4	26-May-13	
CR62	PA14	14-May-13	no dining room
CR56	PG36	28-Apr-13	

Multi-Table Queries

EXAMPLE Simple join

List the names of all clients who have viewed a property, along with any comments supplied.

```
SELECT c.clientNo, fName, lName, propertyNo, comment
FROM Client c, Viewing v
WHERE c.clientNo = v.clientNo;
```

clientNo	fName	lName	propertyNo	comment
CR56	Aline	Stewart	PG36	
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR62	Mary	Tregear	PA14	no dining room
CR76	John	Kay	PG4	too remote

The SQL standard provides the following alternative to specify this join:

```
SELECT c.clientNo, fName, lName, propertyNo, comment
FROM Client c JOIN Viewing v
ON c.clientNo = v.clientNo;
```

Multi-Table Queries

EXAMPLE **Sorting a join**

For branch offices, list the staff numbers and names of staff who manage properties and the properties that they manage.

```
SELECT s.branchNo, s.staffNo, fName, lName, propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
ORDER BY s.branchNo, s.staffNo, propertyNo;
```

In this example, we need to join the Staff and PropertyForRent tables, based on the primary key/foreign key attribute (staffNo). To make the results more readable, we have ordered the output using the branch number as the major sort key and the staff number and property number as the minor keys.

branchNo	staffNo	fName	lName	propertyNo
B003	SG14	David	Ford	PG16
B003	SG37	Ann	Beech	PG21
B003	SG37	Ann	Beech	PG36
B005	SL41	Julie	Lee	PL94
B007	SA9	Mary	Howe	PA14

EXAMPLE Three-table join

List the staff numbers and names of staff who manage properties, including the city in which the branch is located and the properties that the staff manage.

Branch

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

Multi-Table Queries

EXAMPLE **Three-table join**

List the staff numbers and names of staff who manage properties, including the city in which the branch is located and the properties that the staff manage.

```
SELECT b.branchNo, b.city, s.staffNo, fName, IName, propertyNo
FROM Branch b, Staff s, PropertyForRent p
WHERE b.branchNo = s.branchNo AND s.staffNo = p.staffNo
ORDER BY b.branchNo, s.staffNo, propertyNo;
```

branchNo	city	staffNo	fName	IName	propertyNo
B003	Glasgow	SG14	David	Ford	PG16
B003	Glasgow	SG37	Ann	Beech	PG21
B003	Glasgow	SG37	Ann	Beech	PG36
B005	London	SL41	Julie	Lee	PL94
B007	Aberdeen	SA9	Mary	Howe	PA14

Multi-Table Queries

EXAMPLE **Multiple grouping columns**

Find the number of properties handled by each staff member, along with the branch number of the member of staff.

```
SELECT s.branchNo, s.staffNo, COUNT(*) AS myCount
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo;
```

To list the required numbers, we first need to find out which staff actually manage properties. This can be found by joining the Staff and PropertyForRent tables on the staffNo column, using the FROM/WHERE clauses. Next, we need to form groups consisting of the branch number and staff number, using the GROUP BY clause.

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

Computing a Join

□ Conceptually, the procedure for generating the results of a SELECT with a join is as follows:

- 1) Form the Cartesian product of the tables named in the FROM clause.
- 2) Apply the search condition from the WHERE clause to each row of the product table, retaining those rows that satisfy the condition.
- 3) For each remaining row, determine the value of each item in the SELECT list to produce a single row in the result table.
- 4) If SELECT DISTINCT has been specified, eliminate any duplicate rows from the result table.
- 5) If there is an ORDER BY clause, sort the result table as required.

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

- List the carrierID and the distance of all flights landing in NY state airports.

--Subquery

```
SELECT carrierID, distance FROM Flight
WHERE destination IN (SELECT airportID FROM Airport
                      WHERE state = 'NY');
```

--Join

```
SELECT f.carrierID, f.distance
FROM Flight f, Airport a
WHERE f.destination = a.airportID AND a.state = 'NY';
```

Airport

airportID	name	city	state	country
CDG	Charles de Gaulle	Paris		FRA
DXB	Dubai International	Dubai		UAE
JFK	John F Kennedy International	New York	NY	USA
LGA	LaGuardia	New York	NY	USA
MIA	Miami International	Miami	FL	USA
SFO	San Francisco International	San Francisco	CA	USA

Carrier

carrierID	name	fleetSize	employees	revenue
AF	Air France	225	84,602	15,414
DL	Delta Air Lines	855	80,000	39,639
EK	Emirates Airline	239	84,153	18,300
US	US Airways	338	31,467	13,050

Flight

carrierID	flightNo	date	time	airportID	destination	distance
DL	426	10/16/2017	7:00 AM	JFK	SFO	2,586
DL	426	10/17/2017	7:00 AM	JFK	SFO	2,586
DL	479	10/16/2017	7:25 PM	JFK	SFO	2,586
EK	721	10/17/2017	11:25 AM	CDG	DXB	3,259
US	783	10/17/2017	3:20 AM	DXB	JFK	6,843
DL	1472	10/17/2017	7:30 AM	LGA	MIA	1,099
AF	3577	10/17/2017	5:45 PM	JFK	CDG	3,629
US	3577	10/17/2017	9:40 AM	LGA	SFO	2,576
AF	3634	10/16/2017	1:20 PM	CDG	MIA	4,580

- List the names of carriers that have flights landing in France airports and the names of the destination airports.

```
SELECT c.name, a.name
```

```
FROM Carrier c, Flight f, Airport a
```

```
WHERE c.carrierID = f.carrierID AND f.destination = a.airportID
```

```
AND a.country = 'FRA';
```

Outer Joins

- ❑ The join operation combines data from two tables by forming pairs of related rows where the matching columns in each table have the same value.
- ❑ If one row of a table is unmatched, the row is omitted from the result table.
- ❑ The ISO standard provides another set of join operators called outer joins.
- ❑ The Outer join retains rows that do not satisfy the join condition.

Outer Joins

Branch1

branchNo	bCity
B003	Glasgow
B004	Bristol
B002	London

PropertyForRent1

propertyNo	pCity
PA14	Aberdeen
PL94	London
PG4	Glasgow

The (Inner) join of these two tables:

```
SELECT b.*, p.*  
FROM Branch1 b, PropertyForRent1 p  
WHERE b.bCity = p.pCity;
```

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

- ❑ Note that there is no row corresponding to the branch office in Bristol and there is no row corresponding to the property in Aberdeen.
- ❑ If we want to include the unmatched rows, we can use an Outer join.
- ❑ There are three types of Outer join: **Left**, **Right**, and **Full** Outer joins.

Outer Joins

Branch1

branchNo	bCity
B003	Glasgow
B004	Bristol
B002	London

PropertyForRent1

propertyNo	pCity
PA14	Aberdeen
PL94	London
PG4	Glasgow

EXAMPLE Left Outer join

List all branch offices and any properties that are in the same city.

```
SELECT b.*, p.*  
FROM Branch1 b LEFT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

- ❑ The Left Outer join includes not only those rows that have the same city, but also those rows of the first (left) table that are unmatched with rows from the second (right) table. The columns from the second table are filled with NULLs.

Outer Joins

Branch1

branchNo	bCity
B003	Glasgow
B004	Bristol
B002	London

PropertyForRent1

propertyNo	pCity
PA14	Aberdeen
PL94	London
PG4	Glasgow

EXAMPLE Right Outer join

List all properties and any branch offices that are in the same city.

```
SELECT b.*, p.*  
FROM Branch1 b RIGHT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

- ❑ The Right Outer join includes not only those rows that have the same city, but also those rows of the second (right) table that are unmatched with rows from the first (left) table. The columns from the first table are filled with NULLs.

Outer Joins

Branch1

branchNo	bCity
B003	Glasgow
B004	Bristol
B002	London

PropertyForRent1

propertyNo	pCity
PA14	Aberdeen
PL94	London
PG4	Glasgow

EXAMPLE Full Outer join

List the branch offices and properties that are in the same city along with any unmatched branches or properties.

SELECT b.*, p.*

FROM Branch1 b **FULL JOIN** PropertyForRent1 p **ON** b.bCity = p.pCity;

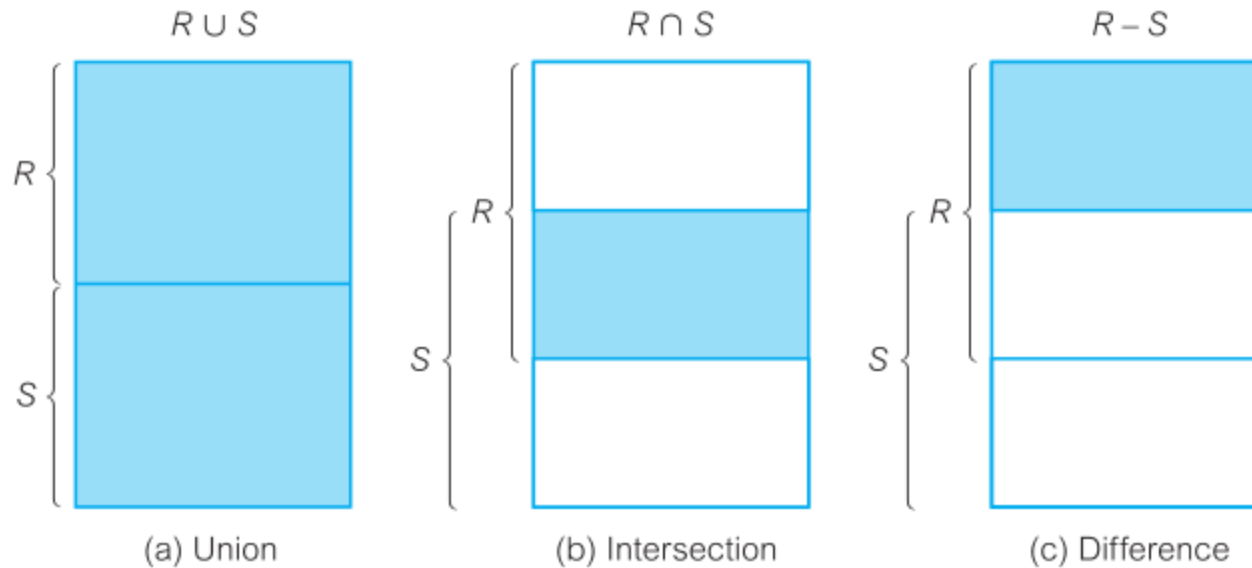
branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

- ❑ The Full Outer join includes not only those rows that have the same city, but also those rows that are unmatched in both tables. The unmatched columns are filled with NULLs.

Combining Result Tables

- ❑ In SQL, we can use the normal set operations of *Union*, *Intersection*, and *Difference* to combine the results of two or more queries into a single result table:
 - The **Union** of two tables, A and B, is a table containing all rows that are in both tables A and B.
 - The **Intersection** of two tables, A and B, is a table containing all rows that are common to both tables A and B.
 - The **Difference** of two tables, A and B, is a table containing all rows that are in table A but are not in table B.
- ❑ The tables have to be **union-compatible**; that is, they have the same structure.

Combining Result Tables



- ❑ The three set operators in the ISO standard are called UNION, INTERSECT, and EXCEPT (or MINUS).
- ❑ If ALL is specified with UNION, the result can include duplicate rows.

Combining Result Tables

Use of UNION

Construct a list of all cities where there is either a branch office or a property.

```
SELECT city  
FROM Branch  
UNION  
SELECT city  
FROM PropertyForRent ;
```

city

London

Glasgow

Aberdeen

Bristol

- ❑ This query is executed by producing a result table from the first query and a result table from the second query, and then merging both tables into a single result table consisting of all the rows from both result tables with the duplicate rows removed.

Combining Result Tables

Use of INTERSECT

Construct a list of all cities where there is both a branch office and a property.

```
SELECT city  
FROM Branch  
INTERSECT  
SELECT city  
FROM PropertyForRent ;
```

city

Aberdeen

Glasgow

London

- ❑ This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that are common to both result tables.
- ❑ This query can be rewritten using a join or a subquery.

Combining Result Tables

Use of EXCEPT

Construct a list of all cities where there is a branch office but no properties.

```
SELECT city  
FROM Branch  
EXCEPT  
SELECT city  
FROM PropertyForRent ;
```

city

Bristol

- ❑ This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that appear in the first result table but not in the second one.

Database Updates

- ❑ SQL is a complete data manipulation language that can be used for modifying the data in the database as well as querying the database.
- ❑ The commands for modifying the database are not as complex as the SELECT statements.
- ❑ In this section, we describe the three SQL statements that are available to modify the contents of the tables in the database:
 - INSERT – adds new rows of data to a table
 - UPDATE – modifies existing data in a table
 - DELETE – removes rows of data from a table

Adding Data to the Database (INSERT)

❑ 1st Form:

INSERT INTO TableName [(columnList)] **VALUES** (dataValueList)

- *TableName* may be either a base table or an updatable view.
- *columnList* represents a list of one or more column names separated by commas.
- The *dataValueList* must match the *columnList* as follows:
 - The number of items in each list must be the same.
 - There must be a direct correspondence in the position of items in the two lists.
 - The data type of each item in *dataValueList* must be compatible with the data type of the corresponding column.

- ❑ The *columnList* is optional; if omitted, SQL assumes a list of all columns in their original CREATE TABLE order.

Adding Data to the Database (INSERT)

INSERT ... VALUES

Insert a new row into the Staff table supplying data for all columns.

```
INSERT INTO Staff  
VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M', '1957-05-25', 8300, 'B003');
```

- ❑ As we are inserting data into each column in the order the table was created, there is no need to specify a column list.
- ❑ Note that character literals such as 'Alan' must be enclosed in single quotes.

Adding Data to the Database (INSERT)

INSERT using defaults

Insert a new row into the Staff table supplying data for all mandatory columns: staffNo, fName, lName, position, salary, and branchNo.

```
INSERT INTO Staff (staffNo, fName, lName, position, salary, branchNo)
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', 8100, 'B003');
```

- ❑ As we are inserting data into only certain columns, we must specify the names of the columns that we are inserting data into.
- ❑ We could also express the INSERT statement as:

```
INSERT INTO Staff
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', NULL, NULL, 8100, 'B003');
```

Adding Data to the Database (INSERT)

□ 2nd Form:

INSERT statement allows multiple rows to be copied from one or more tables to another:

```
INSERT INTO TableName [(columnList)]  
SELECT . . .
```

- *TableName* and *columnList* are defined as before when inserting a single row.
- The **SELECT** clause can be any valid **SELECT** statement.
- The rows inserted into the named table are identical to the result table produced by the *subselect*.
- The same restrictions that apply to the first form of the **INSERT** statement also apply here.

Adding Data to the Database (INSERT)

INSERT ... SELECT

Assume the table: StaffPropCount(staffNo, fName, lName, propCount)

Populate the StaffPropCount table using details from the Staff and PropertyForRent tables.

```
INSERT INTO StaffPropCount
SELECT s.staffNo, fName, lName, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.staffNo, fName, lName;
```

staffNo	fName	lName	propCount
SG14	David	Ford	1
SL41	Julie	Lee	1
SG37	Ann	Beech	2
SA9	Mary	Howe	1

Modifying Data in the Database (UPDATE)

- ❑ The UPDATE statement allows the contents of existing rows in a named table to be changed.

UPDATE TableName

SET columnName1 = dataValue1 [, columnName2 = dataValue2 . . .]

[WHERE searchCondition]

- *TableName* can be the name of a base table or an updatable view.
- The SET clause specifies the names of one or more columns that are to be updated.
- The WHERE clause is optional:
 - If specified, only those rows that satisfy the *searchCondition* are updated.
 - If omitted, the named columns are updated for all rows in the table.
- The new *dataValue(s)* must be compatible with the data type(s) for the corresponding column(s).

EXAMPLE UPDATE all rows

Give all staff a 3% pay increase.

```
UPDATE Staff  
SET salary = salary*1.03;
```

As the update applies to all rows in the Staff table, the WHERE clause is omitted.

EXAMPLE UPDATE specific rows

Give all Managers a 5% pay increase.

```
UPDATE Staff  
SET salary = salary*1.05  
WHERE position = 'Manager';
```

The WHERE clause finds the rows that contain data for Managers and the update salary = salary*1.05 is applied only to these particular rows.

EXAMPLE UPDATE multiple columns

Promote David Ford (staffNo = 'SG14') to Manager and change his salary to £18,000.

```
UPDATE Staff  
SET position = 'Manager', salary = 18000  
WHERE staffNo = 'SG14';
```

Updating a Table Using Another Table

- Update the manager column of the **Branch** table with the appropriate manager name.

Branch

branchNo	address	city	manager
B005	22 Deer Rd	London	<i>NULL</i>
B007	16 Argyll St	Aberdeen	<i>NULL</i>
B003	163 Main St	Glasgow	<i>NULL</i>
B004	32 Manse Rd	Bristol	<i>NULL</i>
B002	56 Clover Dr	London	<i>NULL</i>

Staff

staffNo	name	position	sex	salary	branchNo
SL21	John White	Manager	M	30000	B005
SG37	Ann Beech	Assistant	F	12000	B003
SG14	David Ford	Supervisor	M	18000	B003
SA9	Mary Howe	Assistant	F	9000	B007
SG5	Susan Brand	Manager	F	24000	B003
SL41	Julie Lee	Manager	F	9000	B007

UPDATE Branch

SET manager = (**SELECT** name

FROM Staff

WHERE Staff.branchNo = Branch.branchNo

AND position = 'Manager');

branchNo	address	city	manager
B005	22 Deer Rd	London	John White
B007	16 Argyll St	Aberdeen	Julie Lee
B003	163 Main St	Glasgow	Susan Brand
B004	32 Manse Rd	Bristol	<i>NULL</i>
B002	56 Clover Dr	London	<i>NULL</i>

Deleting Data from the Database (DELETE)

- ❑ The DELETE statement allows rows to be deleted from a named table.

DELETE FROM TableName
[**WHERE** searchCondition]

- *TableName* can be the name of a base table or an updatable view.
- The *searchCondition* is optional:
 - If specified, only those rows that satisfy the condition are deleted.
 - If omitted, all rows are deleted from the table.
- This does not delete the table itself.
To delete the table contents and the table definition, the statement **DROP TABLE** must be used instead.

Deleting Data from the Database (DELETE)

EXAMPLE **DELETE specific rows**

Delete all viewings that relate to property PG4.

```
DELETE FROM Viewing  
WHERE propertyNo = 'PG4';
```

The WHERE clause finds the rows for property PG4 and the delete operation is applied only to these particular rows.

EXAMPLE **DELETE all rows**

Delete all rows from the Viewing table.

```
DELETE FROM Viewing;
```

No WHERE clause has been specified, so the delete operation applies to all rows in the table. This query removes all rows from the table, leaving only the table definition, so that we are still able to insert data into the table at a later stage.